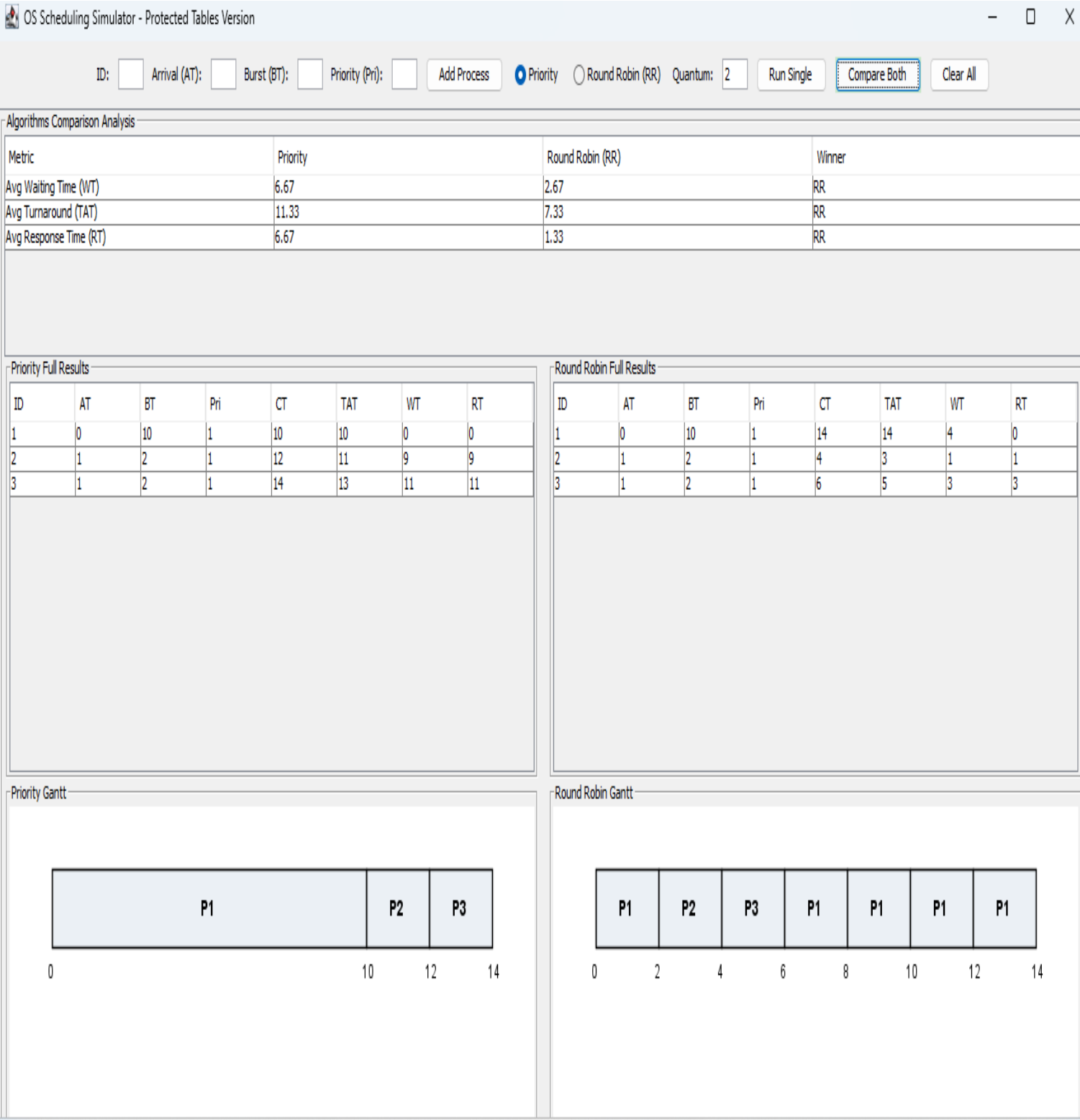


OS Documentation

Round Robin VS Priority

Scenario A: Basic mixed workload ➡



1-Inputs :

P1 : AT=0 , BT=10 , Pri=1

P2 : AT=1 , BT=2 , Pri=1

P3 : AT=1 , BT=2 , Pri=1

Quantum=2

2-Output:

TAT = Completion Time (CT) – Arrival Time (AT)

WT = Turnaround Time (TAT) - Burst Time (BT)

RT = First Start Time - Arrival Time (AT)

For Priority:

➔ P1:

- CT = 10

- TAT = 10 - 0 = 10

- WT = 10 - 10 = 0

- RT = 0 - 0 = 0

➔ P2:

- CT = 12

- TAT = 12 - 1 = 11

- WT = 11 - 2 = 9

- RT = 10 - 1 = 9

➔ P3:

- CT = 14

- TAT = 14 - 1 = 13

- WT = 13 - 2 = 11

- RT = 12 - 1 = 11

-WT =(0 + 9 + 11) / 3 = 20 / 3 = 6.67

-TAT= (10 + 11 + 13) / 3 = 34 / 3 = 11.33

- RT=(0 + 9 + 11) / 3 = 20 / 3 = 6.67

For RR:

➔ P1:

- CT = 14

- TAT = 14 - 0 = 14

- WT = 14 - 10 = 4

- RT = 0 - 0 = 0

➔ P2:

- CT = 4

- TAT = 4 - 1 = 3

- WT = 3 - 2 = 1

- RT = 2 - 1 = 1

➔ P3:

- CT = 6

- TAT = 6 - 1 = 5

- WT = 5 - 2 = 3

- RT = 4 - 1 = 3

-WT = $(4 + 1 + 3) / 3 = 8 / 3 = 2.67$

-TAT = $(14 + 3 + 5) / 3 = 22 / 3 = 7.33$

-RT = $(0 + 1 + 3) / 3 = 4 / 3 = 1.33$

3-Analysis:

-In this scenario, we tested how both algorithms handle a long process (P1) followed by short processes (P2, P3) with the same priority

-RR is winner in WT , TAT , RT because used Quantum time for all process (this leads to fairness process)

-Priority is loser because he used priority (Even if the process takes a long time) This means that a process might takes along time , like the initial process in this example , and the smaller processes might have to wait

4- Conclusion: This test proves that **RR** is the best choice for achieving **Fairness** and preventing short processes from being delayed by long ones.

Scenario B: Urgency case ➔

OS Scheduling Simulator - Protected Tables Version

ID:

Arrival (AT):

Burst (BT):

Priority (Pri):

Add Process

☒ Priority

☐ Round Robin (RR)

Quantum:

Run Single

Compare Both

Clear All

Algorithms Comparison Analysis

Metric	Priority	Round Robin (RR)	Winner
Avg Waiting Time (WT)	4.33	2.67	RR
Avg Turnaround (TAT)	9.00	7.33	RR
Avg Response Time (RT)	3.67	1.33	RR

Priority Full Results

ID	AT	BT	Pri	CT	TAT	WT	RT
1	0	10	3	12	12	2	0
2	1	2	1	3	2	0	0
3	1	2	3	14	13	11	11

Round Robin Full Results

ID	AT	BT	Pri	CT	TAT	WT	RT
1	0	10	3	14	14	4	0
2	1	2	1	4	3	1	1
3	1	2	3	6	5	3	3

Priority Gantt

P1

P2

P1

P3

0

1

3

12

14

Round Robin Gantt

P1

P2

P3

P1

P1

P1

P1

0

2

4

6

8

10

12

14

1-Inputs :

P1 : AT=0 , BT=10 , Pri=3

P2 : AT=1 , BT=2 , Pri=1

P3 : AT=1 , BT=2 , Pri=3

Quantum=2

2-Outputs:

TAT = Completion Time (CT) – Arrival Time (AT)

WT = Turnaround Time (TAT) - Burst Time (BT)

RT = First Start Time - Arrival Time (AT)

For Priority:

➔P1:

- CT = 12

- TAT = 12 - 0 = 12

- WT = 12 - 10 = 2

- RT = 0 - 0 = 0

➔ P2:

- CT = 3

- TAT = 3 - 1 = 2

- WT = 2 - 2 = 0

- RT = 1 - 1=0

➔ P3:

- CT = 14

- TAT = 14 - 1 = 13

- WT = 13 - 2 = 11

- RT = 12 - 1 = 11

-WT = (2 + 0 + 11) / 3 = 13 / 3 = 4.33

-TAT= (12 + 2 + 13) / 3 = 9.00

- RT=(0 + 0 + 11) / 3 = 11 / 3 = 3.67

For RR:

TAT = Completion Time (CT) – Arrival Time (AT)

WT = Turnaround Time (TAT) - Burst Time (BT)

RT = First Start Time - Arrival Time (AT)

➔ P1:

- CT = 14

- TAT = 14 - 0 = 14

- WT = 14 - 10 = 4

- RT = 0 - 0 = 0

➔ P2:

- CT = 4

- TAT = 4 - 1 = 3

- WT = 3 - 2 = 1

- RT = 2 - 1 = 1

➔ P3:

- CT = 6

- $TAT = 6 - 1 = 5$

- $WT = 5 - 2 = 3$

- $RT = 4 - 1 = 3$

- $WT = (4 + 1 + 3) / 3 = 8 / 3 = 2.67$

- $TAT = (14 + 3 + 5) / 3 = 22 / 3 = 7.33$

- $RT = (0 + 1 + 3) / 3 = 4 / 3 = 1.33$

3-Analysis:

-In this scenario, we tested how both algorithms handle a High-Priority Short Process (P2) that arrives while a Low-Priority Long Process (P1) is already running.

-RR is winner as a mathematical in WT , TAT , RT because used Quantum time for all process (this leads to fairness process)

-Priority is loser because he used priority (Even if the process takes a long time) , but in this case, he win for critical task because he finished the process that couldn't wait and had a high priority → this means when a process with a higher priority arrived , he left the process he was working on and went to finish the other process that had the higher priority

4- Conclusion: This test proves that **RR** is the best choice for achieving **Fairness** and priority is suitable for critical task

Scenario C: Fairness case ➔

OS Scheduling Simulator - Protected Tables Version

ID:

Arrival (AT):

Burst (BT):

Priority (Pri):

Add Process

☒ Priority

☐ Round Robin (RR)

Quantum:

Run Single

Compare Both

Clear All

Algorithms Comparison Analysis

Metric	Priority	Round Robin (RR)	Winner
Avg Waiting Time (WT)	6.00	10.00	Priority
Avg Turnaround (TAT)	12.00	16.00	Priority
Avg Response Time (RT)	6.00	2.00	RR

Priority Full Results

ID	AT	BT	Pri	CT	TAT	WT	RT
1	0	6	1	6	6	0	0
2	0	6	1	12	12	6	6
3	0	6	1	18	18	12	12

Round Robin Full Results

ID	AT	BT	Pri	CT	TAT	WT	RT
1	0	6	1	14	14	8	0
2	0	6	1	16	16	10	2
3	0	6	1	18	18	12	4

Priority Gantt

P1

P2

P3

061218

Round Robin Gantt

P1

P2

P3

P1

P2

P3

P1

P2

P3

024681012141618

1-Inputs :

P1 : AT=0 , BT=6 , Pri=1

P2 : AT=0 , BT=6 , Pri=1

P3 : AT=0 , BT=6 , Pri=1

Quantum=2

2-Output:

TAT = Completion Time (CT) – Arrival Time (AT)

WT = Turnaround Time (TAT) - Burst Time (BT)

RT = First Start Time - Arrival Time (AT)

For Priority:

➔P1:

- CT = 6

- TAT = 6 - 0 = 6

- WT = 6 - 6 = 0

- RT = 0 - 0 = 0

➔ P2:

- CT = 12

- TAT = 12 - 0 = 12

- WT = 12 - 6 = 6

- RT = 6 - 0 = 6

➔ P3:

- CT = 18

- TAT = 18 - 0 = 18

- WT = 18 - 6 = 12

- RT = 12 - 0 = 12

-WT=(0 + 6 + 12) / 3 = 6.00

-TAT= (6 + 12 + 18) / 3 = 12.00

- RT=(0 + 6 + 12) / 3 = 6.00

For RR:

TAT = Completion Time (CT) – Arrival Time (AT)

WT = Turnaround Time (TAT) - Burst Time (BT)

RT = First Start Time - Arrival Time (AT)

➔ P1:

- CT = 14

- TAT = 14 - 0 = 14

- WT = 14 - 6 = 8

- RT = 0 - 0 = 0

➔ P2:

- CT = 16

- TAT = 16 - 0 = 16

- WT = 16 - 6 = 10

- RT = 2 - 0 = 2

➔ P3:

- CT = 18

- $TAT = 18 - 0 = 18$

- $WT = 18 - 6 = 12$

- $RT = 4 - 0 = 4$

- $WT = (8 + 10 + 12) / 3 = 10.00$

- $TAT = (14 + 16 + 18) / 3 = 16.00$

- $RT = (0 + 2 + 4) / 3 = 2.00$

3-Analysis:

-In this scenario, we tested how both algorithms handle processes are managed with the same priority , arrival time , and burst time

-RR is winner in RT because used Quantum time for all process (this leads to fairness process) , priority winner in Wt ,TAT

-Priority is the mathematical winner in WT and TAT, but it failed in fairness , because the CPU dedicated to the first process , even though the second and third process had the same priority , Arrival time, and Burst time

4- Conclusion: This test proves that RR is the best choice for achieving Fairness and priority is suitable for critical tasks , but when **priority , Arrival time, and Burst time**

Are the same in processes , it will not be suitable , and RR will be the suitable in this case

Scenario D: Possible starvation case ➡

OS Scheduling Simulator - Protected Tables Version

ID:

Arrival (AT):

Burst (BT):

Priority (Pri):

Add Process

☒ Priority ☐ Round Robin (RR)

Quantum:

Run Single

Compare Both

Clear All

Algorithms Comparison Analysis

Metric	Priority	Round Robin (RR)	Winner
Avg Waiting Time (WT)	3.75	8.00	Priority
Avg Turnaround (TAT)	8.50	12.75	Priority
Avg Response Time (RT)	1.50	2.00	Priority

Priority Full Results

ID	AT	BT	Pri	CT	TAT	WT	RT
1	0	10	8	19	19	9	0
2	1	3	1	4	3	0	0
3	2	3	1	7	5	2	2
4	3	3	1	10	7	4	4

Round Robin Full Results

ID	AT	BT	Pri	CT	TAT	WT	RT
1	0	10	8	19	19	9	0
2	1	3	1	11	10	7	1
3	2	3	1	12	10	7	2
4	3	3	1	15	12	9	5

Priority Gantt

P1P2P3P4P1

01471019

Round Robin Gantt

P1P2P3P1P4P2P3P1P4P1P1

0246810111214151719

1-Inputs :

P1 : AT=0 , BT=10 , Pri=8

P2 : AT=1 , BT=3 , Pri=1

P3 : AT=2 , BT=3 , Pri=1

P4: AT=3 , BT=3 , Pri=1

Quantum=2

2-Output:

TAT = Completion Time (CT) – Arrival Time (AT)

WT = Turnaround Time (TAT) - Burst Time (BT)

RT = First Start Time - Arrival Time (AT)

For Priority:

➔P1:

- CT = 19

- TAT = 19 - 0 = 19

- WT = 19 - 10 = 9

- RT = 0 - 0 = 0

➔ P2:

- CT = 4

- TAT = 4 - 1 = 3

- WT = 3 - 3 = 0

- RT = 1 - 1 = 0

➔ P3:

- $CT = 7$

- $TAT = 7 - 2 = 5$

- $WT = 5 - 3 = 2$

- $RT = 4 - 2 = 2$

➔ P4:

- $CT = 10$

- $TAT = 10 - 3 = 7$

- $WT = 7 - 3 = 4$

- $RT = 7 - 3 = 4$

- $WT = (9 + 0 + 2 + 4) / 4 = 15/4 = 3.75$

- $TAT = (19 + 3 + 5 + 7) / 4 = 34/4 = 8.50$

- $RT = (0 + 0 + 2 + 4) / 4 = 6/4 = 1.50$

For RR:

$TAT = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$

$WT = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

$RT = \text{First Start Time} - \text{Arrival Time (AT)}$

➔ P1:

- $CT = 19$

- $TAT = 19 - 0 = 19$

- $WT = 19 - 10 = 9$

- $RT = 0 - 0 = 0$

➔ P2:

- $CT = 11$

- $TAT = 11 - 1 = 10$

- $WT = 10 - 3 = 7$

- $RT = 2 - 1 = 1$

➔ P3:

- $CT = 12$

- $TAT = 12 - 2 = 10$

- $WT = 10 - 3 = 7$

- $RT = 4 - 2 = 2$

➔ P4:

- $CT = 15$

- $TAT = 15 - 3 = 12$

- $WT = 12 - 3 = 9$

- $RT = 8 - 3 = 5$

- $WT = (9 + 7 + 7 + 9) / 4 = 8.00$

- $TAT = (19 + 10 + 10 + 12) / 4 = 51/4 = 12.75$

- $RT = (0 + 1 + 2 + 5) / 4 = 2.00$

3-Analysis:

-In this scenario, we tested how both algorithms handle a Possible Starvation Case. We created a workload where a low-priority process (P1) is interrupted and blocked by multiple high-priority processes (P2, P3, P4)

-Priority is winner as mathematical in WT , TAT , RT , but RR is the winner in this case to prevent starvation with used Quantum time for all process (this leads to fairness process)

4- Conclusion: This test proves that **RR is better than Priority** for achieving **Fairness** and preventing starvation in this case

Scenario E: Validation case ➔

OS Scheduling Simulator - Protected Tables Version

ID: Arrival (AT): Burst (BT): Priority (Pri): ☒ Priority ☐ Round Robin (RR) Quantum:

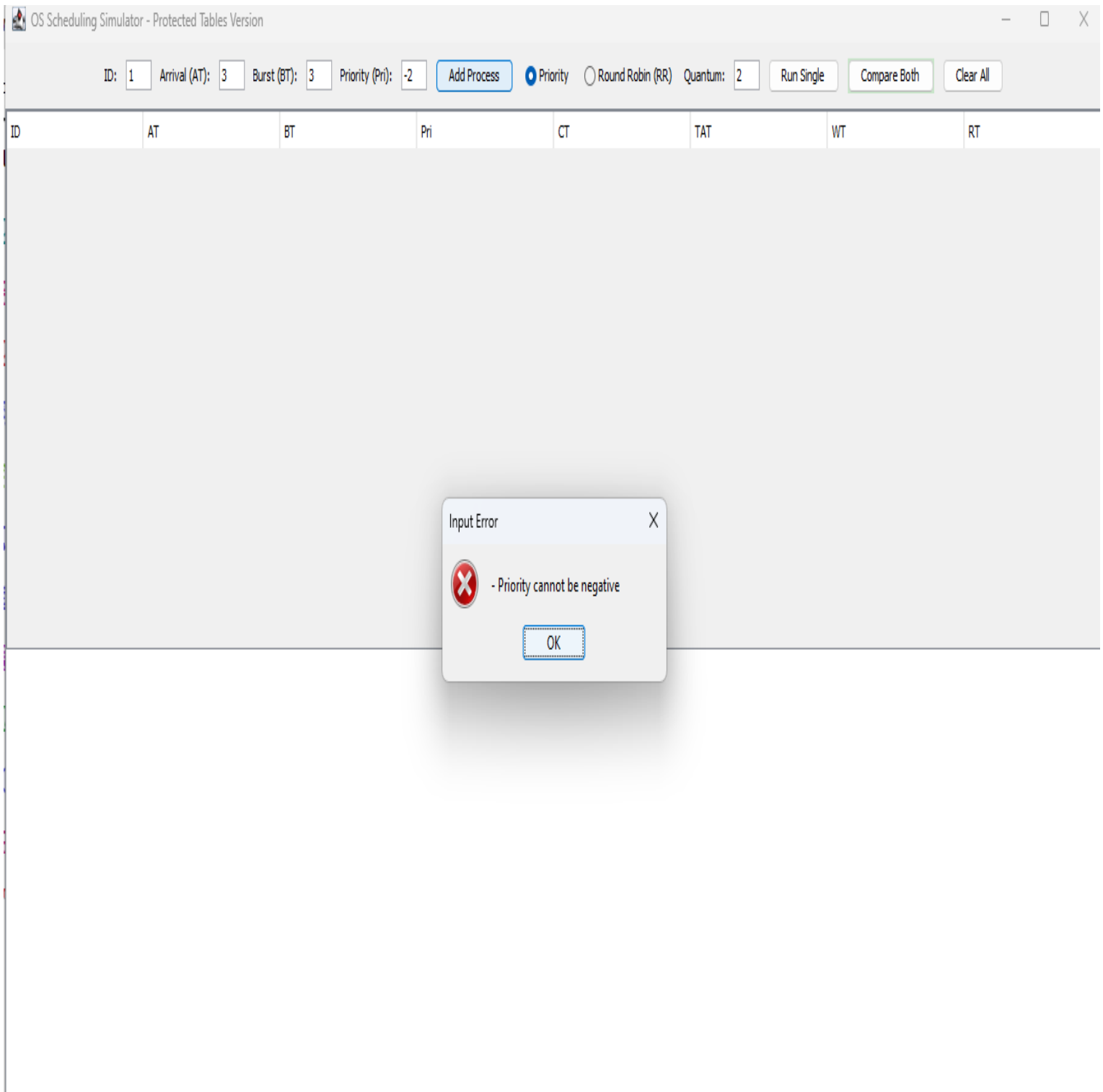
ID	AT	BT	Pri	CT	TAT	WT	RT
----	----	----	-----	----	-----	----	----

Input Error

 - BT must be 1-500

1-Input: BT = -4 (Invalid)

Analysis : BT must be positive(1 to 500)



1-Input: priority = -2 (Invalid)

Analysis : priority must be positive

Required Analysis Questions➔

1-Which algorithm gave better average waiting time?

RR gave better average WT in scenario A (2.67) And scenario B (2.67)

Priority gave better average WT in scenario C (6.00) And scenario D(3.75)

2- Which algorithm gave better response time?

RR gave better RT in scenario A (1.33) , scenario B (1.33) And scenario C (2.00)

Priority gave better RT in scenario D(1.50)

3-Did higher-priority processes gain significant advantage?

in priority algorithm ➔ yes , higher priority gain significant advantage as in scenario B , D

in RR algorithm ➔ no , higher priority doesn't have any significant advantage because all processes have the same time

4- Did Round Robin appear more balanced across all processes?

Yes, RR appear balanced because gave processes the same time Quantum this mean that RR fair to processes as scenario A ,B ,C , D

5- Was starvation observed or likely in Priority Scheduling?

Yes , in scenario D ➔ p1 arrived first (T=0), but when p2 , p3,p4 arrived with higher priority than p1

P2,p3,p4 finished first although p1 arrived first

6-Which algorithm would you recommend for the tested workload, and why?

I would recommend RR , because RR balances the processes and gave them the same time (Quantum) if process doesn't finish , it goes back to the end of the queue

Required Conclusion →

1-State which algorithm performed better on the selected datasets?

No algorithm is better

If you need to balance with processes → RR

If you need to finish processes based on priority (critical tasks) → priority

2- State whether priority-based service improved urgent-task treatment?

Yes, priority scheduling improve urgent-task because when processes arrived with a higher priority than process that CPU works on it , CPU leaves that process and works on the process with higher priority this meaning → preemptive

3- State whether Round Robin improved fairness?

Yes , RR improved fairness , because RR gave specific Quantum time for each process , and any process that doesn't finish within that time must wait until the end of the queue this ensures fairness between processes , no process takes more time with CPU than another

4- State whether starvation risk appeared?

Yes , that appear in scenario D → p1 waits CPU finish p2,p3,p4 although p1 arrived first , but CPU finished p2,p3,p4 because that priority is higher → in priority algorithm